

Operating Systems

Lab experience - signals

Mirco Marchetti, Riccardo Lancellotti

SECloud research group
University of Modena and Reggio Emilia

AY 2025/26

Goals of this lab experience:

- Issue signals from the command
- Develop signal handlers

Warm-up exercises

- Many system programs use signals to perform something useful
- Example: `dd`
 - `dd` is a command line utility for copying data from one file/device to another
 - It can be used to create a file of a specific size filled with random data, for example with the command

```
dd if=/dev/random of=random_file bs=1 count=100000000
```

you will read from `/dev/random` and write to `random_file` 100 million bytes (100 MB) of random data, using a block size of 1 byte (this is very inefficient, it is better to use larger blocks, but we want it to take some time to complete...)
- Execute this command in a terminal, and while it is running, open another terminal and use the `kill` command to send the signal `SIGUSR1` to the `dd` process. It will react by updating you on the progress of the copy operation, without terminating.

- To know the `pid` of the `dd` process, you can execute one of the following commands in a different terminal, while `dd` is running:
 - `pgrep dd`
 - `pidof dd`
 - `ps a`
 - `top`
 - ...
 - Take a look at their manual pages to understand how they work and which one you prefer
- To send the `USR1` signal to the `dd` process, you can execute the following command in a different terminal, while `dd` is running
`kill -SIGUSR1 <pid>`

- To review how to write a signal handler in C, and how to use the `sigaction` system call to register it, you can download and compile the `warmup.c` program. You will find it in Moodle, within the `Laboratorio segnali` folder.
- Read it, compile it, execute it and check its output. Does everything check out? **If not, ask for help to the teaching staff!**

- flags are declared as `static volatile sig_atomic_t`. What does it mean?
- `static` means that the variable is only visible within the file where it is declared. This is a good practice to avoid name clashes with other variables in other files, and to make it clear that the variable is only used within this file.
- `volatile` means that the variable can be changed at any time, without any action being taken by the code the compiler finds nearby. This is important for variables that can be modified by a signal handler.
- `sig_atomic_t` is an integer type that is guaranteed to be read and written atomically, which means that it cannot be interrupted by a signal handler while it is being accessed.
- It is a good practice to declare flags that are modified by signal handlers as `volatile sig_atomic_t` to ensure that they are accessed safely and correctly in the presence of signals. `static` is not so important but still a plus. Remember that and follow this approach!

- signal handlers should be as short as possible, and should not perform any non-async-signal-safe operations. Ideally, they should just set a `volatile sig_atomic_t` flag to indicate that a signal has been received, and return immediately.
- The main loop of the program can then check the flags and perform the necessary actions based on which signals have been received. This approach ensures that the program can handle signals safely and efficiently, without risking undefined behavior.

Do-It-Yourself exercises

- The following slides describe a task that you are expected to solve
- Try to solve the task by yourself or in a very small group (no more than three students), using the knowledge you acquired in the lectures and in the warm-up exercises
- If you have doubts, ask for help to the teaching staff
- We will not provide a solution **now**! Candidate solutions for each exercise will be provided in a few days, after you had the chance to solve the exercises by yourself
- You can use the text editor of your choice. The virtual machine includes Visual Studio Code, mousepad, vim and nano
- **NOTE:** learning is your responsibility! If you let ChatGPT or the smarter classmate solve the exercises for you, you will not learn anything!

Warmup not finished yet?

Exercise 1: add an alarm

- Modify `warmup.c` so that it raises an alarm signal (SIGALRM) every 5 seconds, and the signal handler for SIGALRM prints the message “Alarm received” on the standard output.
- Keep following the best practices for signal handlers that we discussed in the warm-up exercises.

Exercise 2: add a test option

- Modify `warmup.c` to add a command line option `-t` that triggers a test mode. In this mode, the program should `raise()` the `SIGUSR1` signal to itself to check that the signal handler for `SIGUSR1` is working correctly.

Prime Watch

Exercise 3: Prime Watch

- Take a look at `exercise-3.c`. This software takes two numbers as command line arguments: `n-start` and `n-end`. It checks all numbers from `n-start` to `n-end` to see if they are prime or not.
- Read it, understand it, run it.
- Now add handlers for the following signals:
 - `SIGUSR1`: it causes the software to print the number of primality tests and the number of primes found so far
 - `SIGUSR2`: it causes the software to print the largest prime found so far
 - `SIGINT`: it completes the primality test that is currently running, then prints the message “SIGINT received, exiting gracefully” and terminates

- Start by managing SIGUSR1, then SIGUSR2 and finally SIGINT
- a call to `is_prime()` might take a long time! Make sure to check for received signals within the factorization loop, every 1000 iterations for example, to ensure that the software can react to signals in a timely manner
- ...this makes it necessary to handle SIGINT in a different way than the other signals, since it requires to complete the current primality test before terminating...

Signaling children

Exercise 4: Signaling children

- Did you like exercise-3? Now let us make it multiprocess!
- Basic behavior
 - the parent forks 10 children
 - the numbers between `n-start` and `n-end` are divided into 10 equal ranges, and each child is responsible for checking the primality of the numbers in its range
 - When each child terminates it prints the final summary
 - The parent waits for all children to terminate
- Now add the following features:
 - When the parent receives a `SIGUSR1` signal, it sends a `SIGUSR1` to all children
 - When a child receives a `SIGUSR1` signal, it prints the current progress
 - When the parent receives a `SIGUSR2` signal, it sends a `SIGUSR2` to all children
 - When a child receives a `SIGUSR2` signal, it prints the largest prime found so far
 - When the parent receives a `SIGINT` signal, it sends a `SIGINT` to all children
 - When a child receives a `SIGINT` signal, it completes the primality test that is currently running, then prints the final summary and terminates

Exercise 4: Hints

- Start with just one child that does all the work
- Add everything required for signal management
- Then generalize it to more than one child
 - The parent can keep an array of the `pid` of each child, to send signals to them when needed
 - When a child return, the parent can mark it as terminated in the array, to avoid sending signals to it when it is already terminated

Are they horses?!?!

Exercise 5: Are they horses?!?!

- Read `race_rr.c`. Try to get an idea of what it does, then compile and run it
- Read it again, and try to figure out what is happening and how signals are used
- Compare it with `race_rand.c`. What is the main difference among the two?